



428 lines (301 loc) · 7.94 KB

Preview

Code

Blame



Raw



We will be walking thourgh basic linux terminal commands

1. Navigation Commands

pwd – Print Working Directory

Shows the current location in the filesystem.

```
pwd
```



Output example:

```
devsaini@ubuntu:~$ pwd
/home/devsaini
devsaini@ubuntu:~$
```

/home/devsaini/Assingment

ls – List Directory Contents

The ls command is used to list files and directories in the current working directory. flag-a list down all the file and folder including the one which are hidden



```
ls -l → Detailed list (permissions, size, date)
ls -a → Shows hidden files (those starting with .)
ls -la → Combined
```

output is :

```
devsaini@ubuntu:~$ ls
c_class      Eperiment.md      Music           starterkit       Videos
Data         h                  Pictures        Templates
Desktop      h.c               Projects        test
Documents    hello.sh           Public          test1
Downloads    komorebi-2.1-64-bit.deb  snap            'Untitled Document 1'
```

cd – Change Directory

Moves into a directory.

```
cd
```



Examples:

```
cd Documents      # Go to Documents
cd ..             # Go up one level
cd /              # Go to root
cd ~              # Go to home directory
```



```
devsaini@ubuntu:~$ cd Documents
devsaini@ubuntu:~/Documents$
```



2. File and Directory Management

mkdir – Make Directory

Creates a new folder.

```
mkdir new_folder
```



output :

```
devsaini@ubuntu:~$ mkdir file
```

touch – Create File

Creates an empty file.

```
touch file.txt
```



output:

```
evsaini@ubuntu:~/file$ touch file.txt
```

cp – Copy Files or Directories

```
cp source.txt destination.txt
```



mv – Move or Rename Files

```
mv oldname.txt newname.txt
```



rm – Remove Files

```
rm file.txt          # Delete file
rm -r folder_name    # Delete folder (recursively)
```



Be careful! There is no undo.

```
devsaini@ubuntu:~/file$ rm sample2.txt
```



3. File Viewing & Editing

cat – View File Contents

Displays content in terminal.

```
cat file.txt
```



Output:

```
devsaini@ubuntu:~/file$ cat sample.txt  
lol
```

nano - Edit Files in Terminal

A basic terminal-based text editor.

```
nano file.txt
```



Use arrows to move
CTRL + O to save
CTRL + X to exit



output :

```
GNU nano 7.2                               New Buffer  
[ Welcome to nano.  For basic help, type Ctrl+G. ]  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line
```

clear - Clears the Terminal

```
clear
```



Shortcut: CTRL + L

✓ 4. System Commands

echo – Print Text

Useful for debugging or scripting.

```
echo "Hello, World!"
```



output :

```
#!/bin/bash  
echo "Hello World!"
```

whoami – Show Current User

```
whoami
```



output :

```
devsaini@ubuntu:~/shellcmds$ whoami  
devsaini
```

man – Manual for Any Command

```
man ls
```



Use `q` to quit the manual.

✓ 5. Searching and Finding

find – Locate Files

```
find . -name "*.txt"
```



 Finds all `.txt` files in current folder and subfolders.

output :

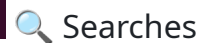
```
devsaini@ubuntu:~/file$ find "sample.txt"
sample.txt
```

grep – Search Inside Files

```
grep "hello" file.txt
```



```
devsaini@ubuntu:~/file$ grep "lol" sample.txt
lol
```



for the word hello inside file.txt .

✓ 6. Helpful Shortcuts

Shortcut	Action
Tab	Auto-complete files/folders
↑ / ↓	Browse command history
CTRL + C	Stop a running command
CTRL + L	Clear screen

✓ 7. Bonus: Chaining Commands

- Run multiple commands:

```
mkdir test && cd test && touch hello.txt
```



- Run only if previous command succeeds: &&
- Run regardless of success: ;

Shell Tutorial – File Permissions with chmod and chown

◆ 1. Understanding File Permissions in Linux

Each file/directory in Linux has:

- **Owner** → The user who created the file.
- **Group** → A group of users who may share access.
- **Others** → Everyone else.

Permission Types

- **r** → Read (4 in numeric)
- **w** → Write (2 in numeric)
- **x** → Execute (1 in numeric)

Permission Layout

Example from `ls -l` :

```
-rwxr-xr--
```



Breakdown:

- `-` → Regular file (`d` = directory, `l` = symlink, etc.)
- `rwx` → Owner has read, write, execute
- `r-x` → Group has read, execute
- `r--` → Others have read only

output :

```
devsaini@ubuntu:~/file/test$ ls -l
total 0
-rw-rw-r-- 1 devsaini devsaini 0 Sep 10 19:22 rachit.txt
```

◆ 2. chmod – Change File Permissions

Syntax

```
chmod [options] mode filename
```



Modes can be set in **numeric (octal)** or **symbolic** form.

(A) Numeric (Octal) Method

Each permission is represented as a number:

- Read = 4
- Write = 2
- Execute = 1

Add them up:

- 7 = rwx
- 6 = rw-
- 5 = r-x
- 4 = r--
- 0 = ---

Example:

```
chmod 777 script.sh
```



Meaning:

- Owner: 7 → rwx
- Group: 7 → r-w-x
- Others: 7 → r-w-x

image



```
devsaini@ubuntu:~/shellcmds$ chmod 777 hello.sh
```

(B) Symbolic Method

Use u (user/owner), g (group), o (others), a (all). Operators:

- + → Add permission
- - → Remove permission
- = → Assign exact permission

Modes can be set in **numeric (octal)** or **symbolic** form.

output:**(C) Recursive Changes**

```
chmod -R 755 /mydir
```



- -R → applies changes recursively to all files/subdirectories.

```
devsaini@ubuntu:~/shellcmds$ chmod -R 755 mydir
```

◆ 3. chown – Change File Ownership**Syntax**

```
chown [options] new_owner:new_group filename
```

**Examples:**

```
chown ashish.txt          # Change owner to user 'sameer'
chown ashish:dev.txt      # Change owner to 'sameer' and group to
chown ashish:dev file.txt # Change only group to 'dev'
chown -R ashish:dev /project # Recursive ownership change
```

**Practice Experiment on chown****◆ 1. Create a new user**

```
sudo useradd -m newuser
```



- -m → creates a home directory /home/newuser .

◆ 2. Create a new group

```
sudo groupadd newgroup
```



◆ 3. Add the user to the group

```
sudo usermod -aG newgroup newuser
```



- -aG → append user to the supplementary group (doesn't remove existing groups).

◆ 4. Create a file (as current user, e.g. root or your login user)

```
touch testfile.txt
```



Check ownership:

```
ls -l testfile.txt
```



Example:

```
-rw-rw-r-- 1 devsaini devsaini 0 Sep 10 13:24 testfile.txt
```



◆ 5. Assign ownership of the file to newuser and newgroup

```
sudo chown newuser:newgroup testfile.txt
```



◆ 6. Verify ownership

```
ls -l testfile.txt
```



Output:

```
-rw-rw-r--1 newuser newgroup 0 Aug 20 18:52 testfile.txt
```



✓ **Key Tip:** Use **numeric** for quick settings (e.g., 755, 644) and **symbolic** for fine adjustments (`u+x` , `g-w`).

```
devsaini@ubuntu:~/shellcmds$ ls -l hello.sh
-rwxrwxrwx 1 devsaini devsaini 33 Sep 10 19:17 hello.sh
```

◆ 4. Putting It All Together

Example Scenario

```
touch project.sh
ls -l project.sh
```



Output:

```
-rw-r--r-- 1 sameer dev 0 Aug 19 12:00 project.sh
```



Now:

```
chmod 700 project.sh      # Only owner has rwx
chmod u+x,g-w project.sh  # Add execute for user, remove write for group
chown root:admin project.sh # Change owner to root and group to admin
```



◆ 5. Quick Reference Table

Numeric	Permission	Meaning
0	---	No access
1	--x	Execute only
2	-w-	Write only
3	-wx	Write + Exec
4	r--	Read only
5	r-x	Read + Exec
6	rw-	Read + Write

Numeric	Permission	Meaning
7	rwx	Full access

Q1 what is the difference between chmod and chown?

ANS=chown-change ownership

change the owner and group of a file or directory



chmod-change permissions

changes the permissions for the owner,group, and others

Q2 how do you check current directory and user?

ANS = by using the pwd command to check current directory and
by using the whoami to check current user.

